

OPERATIONS & DESIGN REFERENCE

Complete Engineering Runbooks, Specs & Design System

School Management Platform · v1.0 · 7 Documents in One

Contents: (1) Database Migration Runbook (2) CI/CD Pipeline Specification (3) Security Audit Checklist (4) OpenAPI Contract (5) UI/UX Design Specification (6) AI Incident Intelligence System (7) Company Owner Portal Design

1 Database Migration Runbook

This runbook defines how the engineering team applies core schema changes across all active school databases — without downtime, without data loss, and with full rollback capability. Core schema changes are changes to the platform's base tables (not school-customized tables), such as adding a new system column, modifying an index, or introducing a new core table.

Critical Distinction

Core migrations (this runbook) affect ALL active schools simultaneously and are deployed by the engineering team. Dynamic migrations (Schema Migration Engine) affect ONE school and are triggered by that school's admin via the canvas. These are two separate systems and must never be confused.

1.1 Migration Versioning

Core migrations are versioned files stored in `/packages/migrations/core/` in the monorepo. Every file is named with a zero-padded sequential version number and a descriptive name. Migrations are immutable once merged to main — never edit an existing migration file.

```
packages/migrations/core/
  001_init_schema.sql
  002_add_students_table.sql
  003_add_branch_support.sql
  004_add_meta_schema_registry.sql
  005_add_workflow_engine_tables.sql
  ...

-- Every migration file must include:
-- 1. A header comment with: version, author, date, description, rollback
instructions
-- 2. The forward migration SQL
-- 3. A corresponding rollback file: 003_add_branch_support.rollback.sql

-- Example: 003_add_branch_support.sql
-- Version:      003
-- Author:       engineering@platform.com
-- Date:         2026-01-15
-- Description:  Add branches table and branch_id FK to students and staff
-- Rollback:     003_add_branch_support.rollback.sql

CREATE TABLE branches (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id   UUID NOT NULL,
  name        TEXT NOT NULL,
  address     TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_branches_school ON branches(school_id);

ALTER TABLE students ADD COLUMN branch_id UUID REFERENCES branches(id);
```

```
CREATE INDEX idx_students_branch ON students(branch_id);
```

1.2 Migration Runner — Algorithm

The Migration Runner is a CLI tool (packages/tools/migrate.ts) that runs as a GitHub Actions step during deployment. It connects to every active school's database and applies any pending migrations sequentially.

```
// tools/migrate.ts – simplified
async function runCoreMigrations(): Promise<void> {
  // 1. Fetch all ACTIVE schools from Control Plane
  const schools = await controlPlaneDB.query(
    `SELECT id, slug, db_secret_id FROM schools WHERE status = 'ACTIVE' ORDER BY
    created_at`
  );
  console.log(`Running migrations on ${schools.length} school databases`);

  // 2. Load all migration files from disk, sorted by version
  const migrations = await loadMigrationFiles('./migrations/core/');

  const results: MigrationResult[] = [];

  // 3. For each school, apply pending migrations
  for (const school of schools) {
    const conn = await getConnection(school.db_secret_id);
    try {
      const applied = await getAppliedVersions(conn); // reads
      _migration_history
      const pending = migrations.filter(m => !applied.includes(m.version));
      if (pending.length === 0) { results.push({school:school.slug,
        status:'SKIP'}); continue; }

      for (const migration of pending) {
        await conn.query('BEGIN');
        try {
          await conn.query(migration.sql);
          await conn.query(
            `INSERT INTO _migration_history(version,name,applied_at)
VALUES($1,$2,now())`,
            [migration.version, migration.name]
          );
          await conn.query('COMMIT');
        } catch(err) {
          await conn.query('ROLLBACK');
          throw err;
        }
      }
      results.push({school:school.slug, status:'OK',
        appliedCount:pending.length});
    } catch(err) {
      results.push({school:school.slug, status:'FAILED', error:String(err)});
      // Do NOT stop – continue to next school, alert ops
      await alertOps({school:school.slug, error:String(err)});
    }
  }
}
```

```

    } finally {
      conn.release();
    }
  }
  // 4. Report summary – any FAILED schools require manual intervention
  await writeMigrationReport(results);
  const failed = results.filter(r => r.status==='FAILED');
  if (failed.length > 0) process.exit(1); // fail CI if any school failed
}

```

1.3 Zero-Downtime Techniques

Technique	Implementation
ADD COLUMN with DEFAULT	In PostgreSQL 16, adding a nullable column or a column with a constant default is instant (metadata-only). No table rewrite. Always add new columns as nullable first, backfill data, then add NOT NULL constraint in a separate migration.
Two-phase NOT NULL	Phase 1: ADD COLUMN nullable. Phase 2 (next deploy): UPDATE table SET col = default WHERE col IS NULL. Phase 3: ALTER COLUMN SET NOT NULL. Never do all three in one migration.
Index creation	Always use CREATE INDEX CONCURRENTLY. Never CREATE INDEX without CONCURRENTLY on a live table — it takes an exclusive lock and blocks all reads/writes for the duration.
Column rename	Never rename a column directly. Add new column → copy data → update app to read new column → drop old column in a future migration after confirming no old code references it.
Table rename	Same pattern as column rename — add new table, migrate data, update app, drop old table. Never ALTER TABLE RENAME on a live production table.
Large backfills	If backfilling data on a table with >50k rows, batch in chunks of 1000 rows with a 50ms sleep between batches to avoid locking. Use a BullMQ job, not inline SQL in the migration.

1.4 Rollback Procedure

```

// Emergency rollback – run against affected school(s)
// Step 1: Identify the failed migration version
SELECT version, name, applied_at FROM _migration_history ORDER BY version DESC
LIMIT 5;

// Step 2: Run the rollback SQL file for that version
-- e.g., for version 003:
\i packages/migrations/core/003_add_branch_support.rollback.sql

// Step 3: Delete the migration history entry
DELETE FROM _migration_history WHERE version = 003;

// Step 4: Verify schema matches expected state
SELECT table_name FROM information_schema.tables WHERE table_schema = 'public'

```

```
ORDER BY table_name;

// Step 5: Alert engineering team, create post-mortem ticket
```

1.5 Migration Failure Scenarios

Failure	Cause	Resolution
Constraint violation	New NOT NULL column added but existing rows have NULL values	Always add as nullable first. Pre-deployment check: run dry-run query counting NULL rows before migration.
Deadlock on large table	Migration locks table, blocking live school operations	Use CONCURRENTLY for indexes. For column additions, PG 16 fast-path makes most additions lock-free.
One school fails	Migration succeeds on 499 schools, fails on 1	Runner continues to other schools. Failed school flagged in report. Ops team manually resolves and re-runs for that school only.
Migration runner crashes	Process killed mid-run	Idempotent design: each migration checks _migration_history before applying. Re-running the runner is always safe.
Disk full on school DB	Cloud SQL instance out of disk during migration	GCP auto-storage-increase enabled on all instances. Alert fires at 80% disk usage. Pre-migration check: verify available disk > 2x estimated migration size.

2 CI/CD Pipeline Specification

All code changes flow through GitHub Actions pipelines. There are three pipeline types: PR pipelines (run on every pull request), Deploy pipelines (run on merge to main), and Rollback pipelines (manually triggered). The monorepo uses Turborepo's dependency graph to only rebuild and test packages that changed.

2.1 Branch Strategy

Branch	Purpose
main	Production branch. Protected — direct pushes blocked. Only merges via approved PRs. Every merge triggers a deploy.
develop	Staging branch. Auto-deploys to staging environment. Used for integration testing before PRs to main.
feature/xxx	Feature branches. All development work. PRs target develop first, then main after QA.
hotfix/xxx	Emergency patches. PRs directly to main with expedited review. Must include rollback plan.
release/x.x	Release branches for versioned releases. Tagged on merge to main.

2.2 PR Pipeline (.github/workflows/pr.yml)

```
name: PR Checks
on:
  pull_request:
    branches: [main, develop]

jobs:
  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22', cache: 'npm' }
      - run: npm ci
      - run: npx turbo typecheck --filter='[HEAD^1]'
        # Only typecheck packages that changed

  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx turbo lint --filter='[HEAD^1]'

  test:
```

```
runs-on: ubuntu-latest
services:
  postgres:
    image: postgres:16
    env: { POSTGRES_PASSWORD: test, POSTGRES_DB: testdb }
    ports: ['5432:5432']
  redis:
    image: redis:7
    ports: ['6379:6379']
steps:
  - uses: actions/checkout@v4
  - run: npm ci
  - run: npx turbo test --filter='[HEAD^1]'
  env:
    DATABASE_URL: postgresql://postgres:test@localhost:5432/testdb
    REDIS_URL: redis://localhost:6379

build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm ci
    - run: npx turbo build --filter='[HEAD^1]'

migration-dry-run:
  runs-on: ubuntu-latest
  if: contains(github.event.pull_request.labels.*.name, 'migration')
  steps:
    - uses: actions/checkout@v4
    - run: npm ci
    - run: npx tsx tools/migrate.ts --dry-run --env=staging
      # Validates all migration SQL without applying
```

2.3 Deploy Pipeline (.github/workflows/deploy.yml)

```
name: Deploy to Production
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    concurrency:
      group: production-deploy
      cancel-in-progress: false # Never cancel a running deploy
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with: { credentials_json: '${{ secrets.GCP_SA_KEY }}' }

      # 1. Build all apps
```

```

- name: Build
  run: npx turbo build

# 2. Run core database migrations on ALL active schools
- name: Run Migrations
  run: npx tsx tools/migrate.ts --env=production
  env:
    CONTROL_PLANE_DB_URL: ${ secrets.PROD_CONTROL_PLANE_DB_URL }
    GCP_PROJECT_ID: ${ secrets.GCP_PROJECT_ID }

# 3. Deploy API to Cloud Run
- name: Deploy API
  run: |
    docker build -t gcr.io/$PROJECT/api:$GITHUB_SHA ./apps/api
    docker push gcr.io/$PROJECT/api:$GITHUB_SHA
    gcloud run deploy api --image gcr.io/$PROJECT/api:$GITHUB_SHA \
      --region asia-south1 --min-instances 2 --max-instances 50 \
      --set-env-vars DEPLOY_SHA=$GITHUB_SHA

# 4. Deploy Web app to Cloud Run
- name: Deploy Web
  run: |
    docker build -t gcr.io/$PROJECT/web:$GITHUB_SHA ./apps/web
    docker push gcr.io/$PROJECT/web:$GITHUB_SHA
    gcloud run deploy web --image gcr.io/$PROJECT/web:$GITHUB_SHA \
      --region asia-south1 --min-instances 1 --max-instances 20

# 5. Deploy Owner Portal
- name: Deploy Owner Portal
  run: |
    docker build -t gcr.io/$PROJECT/portal:$GITHUB_SHA ./apps/owner-portal
    docker push gcr.io/$PROJECT/portal:$GITHUB_SHA
    gcloud run deploy owner-portal --image gcr.io/$PROJECT/portal:
$GITHUB_SHA \
      --region asia-south1 --min-instances 1

# 6. Smoke tests against production
- name: Smoke Tests
  run: npx tsx tools/smoke-test.ts --env=production

# 7. Notify Slack on success
- name: Notify
  if: success()
  run: |
    curl -X POST ${ secrets.SLACK_WEBHOOK } \
      -d '{"text":"Deploy ${ github.sha } succeeded"}'

# 8. Auto-rollback on failure
- name: Rollback on failure
  if: failure()
  run: |
    gcloud run services update-traffic api --to-revisions=LATEST-1=100 --
region asia-south1
    gcloud run services update-traffic web --to-revisions=LATEST-1=100 --

```



```
region asia-south1
  curl -X POST ${ secrets.SLACK_WEBHOOK } \
    -d '{"text":"DEPLOY FAILED – AUTO ROLLED BACK to previous
revision"}'
```

2.4 Mobile Build Pipeline (.github/workflows/mobile.yml)

```
name: Mobile Build
on:
  push:
    branches: [main]
    paths: ['apps/mobile/**', 'packages/**']

jobs:
  ios:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm ci && cd apps/mobile && npx pod-install
      - run: npx react-native build-ios --mode=Release
      - name: Upload to TestFlight
        run: xcrun altool --upload-app -f build/App.ipa \
          --apiKey ${ secrets.APP_STORE_KEY }

  android:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17' }
      - run: npm ci
      - run: cd apps/mobile/android && ./gradlew assembleRelease
      - name: Sign APK
        run: |
          echo ${ secrets.KEYSTORE_BASE64 } | base64 -d > release.keystore
          zipalign -v 4 app-release-unsigned.apk app-release.apk
          apksigner sign --ks release.keystore app-release.apk
      - name: Upload to Play Store
        uses: r0adkll/upload-google-play@v1
        with:
          serviceAccountJsonPlainText: ${ secrets.PLAY_STORE_SA }
          releaseFiles: app-release.apk
          track: internal
```

3 Security Audit Checklist

This checklist defines the complete pre-launch security audit and penetration testing scope. Every item must be verified by an independent security engineer or third-party pen testing firm before the platform accepts real school data.

3.1 Authentication & Session Security

Check	Requirement	Severity
JWT signing algorithm	Verify RS256 (asymmetric) is used, not HS256 (symmetric shared secret)	CRITICAL
Token expiry	Access tokens expire in 15 minutes. Refresh tokens expire in 30 days.	CRITICAL
Refresh token rotation	Each use of refresh token issues a new one and invalidates the old.	CRITICAL
httpOnly cookie flag	Refresh token cookie has httpOnly, Secure, SameSite=Strict flags.	CRITICAL
JWT none algorithm	Server rejects tokens with alg:none header.	CRITICAL
Brute force protection	Login endpoint rate-limited: 10 attempts/min per IP. Lockout after 20 failures.	HIGH
Password hashing	bcrypt with cost factor 12 minimum. No MD5, SHA-1, or plain storage.	CRITICAL
Session invalidation	Logout invalidates refresh token in DB. Password reset invalidates all sessions.	HIGH
2FA availability	TOTP-based 2FA available for super admin accounts.	HIGH

3.2 API Security

Check	Requirement	Severity
SQL injection	All queries use parameterized statements. No string interpolation of user input into SQL.	CRITICAL
Schema name injection	Column/table names from canvas validated against [a-z0-9_] whitelist before use in SQL.	CRITICAL
IDOR (object reference)	Every DB query scoped by school_id from JWT, not from request params.	CRITICAL
Cross-tenant data access	Penetration test: can school A's token read school B's data? Must be	CRITICAL

	impossible.	
Rate limiting bypass	Test rate limits with IP rotation, header spoofing, token cycling.	HIGH
CORS policy	API allows only known frontend origins. Wildcard (*) origin blocked.	HIGH
HTTP security headers	Helmet.js: CSP, HSTS, X-Frame-Options, X-Content-Type-Options all set.	HIGH
File upload validation	Uploaded files scanned with ClamAV. MIME type verified server-side, not client header.	HIGH
WebSocket auth	WS connections require valid JWT in handshake. Unauthenticated WS connections rejected.	CRITICAL
Mass assignment	API routes never accept raw req.body into DB queries. Explicit field allowlists only.	HIGH

3.3 Data Security

Check	Requirement	Severity
Encryption at rest	All Cloud SQL instances have encryption at rest enabled (GCP default AES-256).	CRITICAL
Encryption in transit	TLS 1.3 enforced for all connections. TLS 1.0 and 1.1 disabled.	CRITICAL
Secrets in code	Zero secrets in source code. GitHub secret scanning enabled on repo.	CRITICAL
Secret Manager usage	All credentials (DB, Redis, API keys) retrieved from GCP Secret Manager at runtime.	CRITICAL
PII in logs	Sentry and application logs scrubbed of: names, email, phone, Aadhar, fees, salary.	CRITICAL
Database direct access	Cloud SQL instances on private VPC. No public IP. Only accessible from API Cloud Run.	CRITICAL
Backup encryption	Automated backups encrypted with school-specific KMS key.	HIGH
Data residency	All school data stored in GCP asia-south1 (Mumbai). No data leaves India.	HIGH
Student data classification	Fields containing Aadhar, medical, or financial data flagged in meta-schema. Extra audit log on access.	HIGH

3.4 Infrastructure Security

Check	Requirement	Severity
VPC isolation	API, DB, Redis all on private VPC. Only Cloud Run exposed publicly via Load Balancer.	CRITICAL
IAM least privilege	Each service account has minimum required permissions. No Editor/Owner roles on service accounts.	HIGH
Cloud Run egress	API Cloud Run has VPC connector — outbound traffic goes through VPC, not public internet.	HIGH
DDoS protection	GCP Cloud Armor WAF enabled on Load Balancer. Rate limiting at infrastructure level.	HIGH
Dependency scanning	npm audit and Dependabot alerts enabled. Critical CVEs must be patched within 72 hours.	HIGH
Container image scanning	GCP Artifact Registry scans all Docker images for CVEs before deployment.	HIGH
CI/CD secret handling	GitHub Actions secrets never printed in logs. Actions pinned to specific SHA, not tags.	HIGH
MFA on GCP Console	All GCP project IAM users require MFA. Service account keys rotated every 90 days.	CRITICAL

3.5 Penetration Testing Scope

The following attack scenarios must be explicitly tested by the pen testing firm before launch:

- Attempt to access school B's student records using school A's valid JWT
- Attempt to inject malicious SQL through the canvas schema change API
- Attempt to provision a new school without a valid subscription
- Attempt to call the AI agent tool 'query_school_data' with a SQL injection payload
- Attempt to access the Company Owner Portal without a valid internal session
- Attempt to reset another school's super admin password via the forgot-password flow
- Attempt to upload a malicious file disguised as a PDF (polyglot file attack)
- Attempt to exhaust Cloud SQL connections via API flooding (connection pool attack)
- Attempt to read environment variables or secrets via SSRF through the AI agent tool
- Attempt to break out of tenant isolation via WebSocket namespace spoofing

4 OpenAPI Contract — Core Endpoints

This section defines the API contract for all core endpoints. Full OpenAPI 3.1 YAML spec is auto-generated from Fastify route schemas using fastify-swagger. What follows is the human-readable reference for engineers.

4.1 Authentication

Endpoint	Contract
POST /api/v1/auth/login	Body: { email, password }. Returns: { accessToken, user }. Sets httpOnly refresh token cookie. Rate: 10/min per IP.
POST /api/v1/auth/refresh	No body — reads httpOnly cookie. Returns: { accessToken }. Rotates refresh token.
POST /api/v1/auth/logout	Invalidates refresh token. Clears cookie.
POST /api/v1/auth/forgot-password	Body: { email }. Sends reset link. Rate: 3/min per IP.
POST /api/v1/auth/reset-password	Body: { token, newPassword }. Token expires in 1 hour. Invalidates all sessions.

4.2 School Data — Students

```

GET    /api/v1/school/students
  Query: page, limit, search, classId, branchId, sortBy, sortDir
  Response 200: { data: Student[], meta: { total, page, limit } }
  Permission: READ on 'students'
  Dynamic fields: response includes only fields readable by requesting role's
  permissions

POST   /api/v1/school/students
  Body: StudentCreateInput (all required core fields + any dynamic fields)
  Response 201: { data: Student }
  Permission: CREATE on 'students'

GET    /api/v1/school/students/:id
  Response 200: { data: Student }
  Permission: READ on 'students'

PATCH /api/v1/school/students/:id
  Body: Partial<StudentUpdateInput>
  Response 200: { data: Student }
  Permission: UPDATE on 'students'

DELETE /api/v1/school/students/:id
  Response 200: { success: true }
  Permission: DELETE on 'students'
  Note: Soft delete — sets deleted_at, not physical row deletion
  
```

4.3 Schema Engine Endpoints

```
POST    /api/v1/schema/change
  Body: SchemaChangeDescriptor
  Response 200: { success: true, newVersion: number }
  Response 400: { error: string } -- validation failure
  Response 409: { error: 'Conflict', message: string } -- concurrent edit
  Permission: CANVAS_EDIT
  Rate limit: 10/min per user

GET     /api/v1/schema/meta
  Response 200: { data: MetaSchema }
  Permission: Any authenticated user
  Cache: 60 seconds (ETag-based)

GET     /api/v1/schema/history
  Query: limit (default 20), before (cursor)
  Response 200: { data: MigrationLogEntry[] }
  Permission: SUPER_ADMIN or SCHEMA_VIEWER

POST    /api/v1/schema/rollback
  Body: { targetVersion: number, confirmationToken: string }
  Response 200: { success: true }
  Permission: SUPER_ADMIN only
  Note: confirmationToken is a one-time token issued by GET
  /api/v1/schema/rollback/preview
```

4.4 Canvas Endpoints

```
GET     /api/v1/canvas/:type
  Param: type = 'roles' | 'workflows'
  Response 200: { data: CanvasState }
  Permission: CANVAS_VIEW

POST    /api/v1/canvas/:type/save
  Body: CanvasState
  Response 200: { success: true, savedAt: string }
  Permission: CANVAS_EDIT
  Note: Triggers schema diff + SME if structural changes detected

GET     /api/v1/canvas/:type/history
  Response 200: { data: CanvasSnapshot[] } -- last 50 snapshots
  Permission: CANVAS_VIEW
```

4.5 AI Agent Endpoints

```
POST    /api/v1/ai/chat
  Body: { message: string, history: ConversationTurn[], screenContext:
  ScreenContext }
```

```
Response 200: { reply: string, toolsUsed: string[], actions: AgentAction[] }  
Rate limit: 30/min per user  
Streaming: POST /api/v1/ai/chat/stream returns Server-Sent Events
```

```
POST /api/v1/ai/voice/transcribe  
Body: multipart/form-data { audio: Blob, language: string }  
Response 200: { transcript: string, confidence: number, provider:  
'sarvam'|'whisper' }
```

```
POST /api/v1/ai/voice/synthesize  
Body: { text: string, language: string, voice: string }  
Response: audio/mpeg stream
```

4.6 Standard Error Responses

```
// All errors follow this envelope:  
{  
  "error": "ERROR_CODE",  
  "message": "Human-readable description",  
  "details": {} // optional, validation errors etc  
}  
  
// Common error codes:  
// 400 VALIDATION_ERROR    - request body/query fails schema validation  
// 401 UNAUTHORIZED        - missing or expired JWT  
// 403 FORBIDDEN           - valid JWT but insufficient permissions  
// 404 NOT_FOUND            - resource doesn't exist or not accessible by tenant  
// 409 CONFLICT            - optimistic lock conflict (meta-schema version  
mismatch)  
// 422 SCHEMA_VIOLATION    - schema change descriptor fails business rules  
// 429 RATE_LIMIT_EXCEEDED - too many requests  
// 500 INTERNAL_ERROR       - unexpected server error (logged to Sentry)  
// 503 SERVICE_UNAVAILABLE - DB connection pool exhausted or Cloud SQL  
maintenance
```

5 UI/UX Design Specification

The platform's design system targets Apple and Tesla-level UI quality — calm, fast, intentional. Everything is built in-house. No third-party component libraries. This spec defines tokens, components, canvas interaction patterns, and motion design.

5.1 Design Tokens

Color System

```
/* tokens/colors.css */

:root {
  /* Brand */
  --color-brand-50: #EEF2FF;
  --color-brand-100: #C7D2FE;
  --color-brand-500: #1A3C6B; /* primary */
  --color-brand-600: #142F56; /* hover */
  --color-brand-900: #0A1A30;

  /* Accent */
  --color-accent-400: #FBBF24;
  --color-accent-500: #F59E0B; /* primary accent */
  --color-accent-600: #D97706;

  /* Neutral */
  --color-gray-0: #FFFFFF;
  --color-gray-50: #F9FAFB;
  --color-gray-100: #F3F4F6;
  --color-gray-200: #E5E7EB;
  --color-gray-400: #9CA3AF;
  --color-gray-600: #4B5563;
  --color-gray-900: #111827;

  /* Semantic */
  --color-success: #059669;
  --color-warning: #D97706;
  --color-error: #DC2626;
  --color-info: #2563EB;

  /* Surface */
  --surface-page: var(--color-gray-50);
  --surface-card: var(--color-gray-0);
  --surface-overlay: rgba(0,0,0,0.4);

  /* Text */
  --text-primary: var(--color-gray-900);
  --text-secondary: var(--color-gray-600);
  --text-muted: var(--color-gray-400);
}
```



```
[data-theme='dark'] {  
  --surface-page:    #0F1117;  
  --surface-card:    #1A1D27;  
  --text-primary:    #F9FAFB;  
  --text-secondary:  #9CA3AF;  
}
```

Typography Scale

```
/* tokens/typography.css */  
:root {  
  --font-sans: 'Inter Variable', -apple-system, BlinkMacSystemFont, sans-serif;  
  --font-mono: 'JetBrains Mono', 'Fira Code', monospace;  
  
  /* Scale (rem) */  
  --text-xs: 0.75rem; /* 12px – labels, badges */  
  --text-sm: 0.875rem; /* 14px – secondary text, table data */  
  --text-base: 1rem; /* 16px – body text */  
  --text-lg: 1.125rem; /* 18px – subheadings */  
  --text-xl: 1.25rem; /* 20px – card titles */  
  --text-2xl: 1.5rem; /* 24px – page headings */  
  --text-3xl: 1.875rem; /* 30px – section headings */  
  --text-4xl: 2.25rem; /* 36px – display headings */  
  
  /* Weight */  
  --font-normal: 400;  
  --font-medium: 500;  
  --font-semibold: 600;  
  
  /* Leading */  
  --leading-tight: 1.25;  
  --leading-normal: 1.5;  
  --leading-relaxed: 1.75;  
}
```

Spacing & Radius

```
/* tokens/spacing.css */  
:root {  
  --space-1: 4px;    --space-2: 8px;    --space-3: 12px;  
  --space-4: 16px;   --space-5: 20px;   --space-6: 24px;  
  --space-8: 32px;   --space-10: 40px;  --space-12: 48px;  
  --space-16: 64px;  --space-20: 80px;  --space-24: 96px;  
  
  --radius-sm: 6px;  --radius-md: 10px; --radius-lg: 16px;  
  --radius-xl: 24px; --radius-full: 9999px;  
  
  /* Shadows */  
  --shadow-sm: 0 1px 2px rgba(0,0,0,0.06);  
  --shadow-md: 0 4px 12px rgba(0,0,0,0.08);  
  --shadow-lg: 0 12px 40px rgba(0,0,0,0.12);  
  --shadow-focus: 0 0 0 3px rgba(26,60,107,0.25);  
}
```

Motion Tokens

```
/* tokens/motion.css */
:root {
  /* Durations */
  --dur-instant: 50ms;
  --dur-fast: 100ms;
  --dur-normal: 200ms;
  --dur-slow: 350ms;
  --dur-xslow: 500ms;

  /* Easings */
  --ease-smooth: cubic-bezier(0.4, 0, 0.2, 1); /* material standard */
  --ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1); /* overshoot spring */
  --ease-out: cubic-bezier(0, 0, 0.2, 1); /* decelerate */
  --ease-in: cubic-bezier(0.4, 0, 1, 1); /* accelerate */
}

/* Motion library equivalents for Motion (Framer) */
// spring: { type:'spring', stiffness:300, damping:30 }
// smooth: { duration:0.2, ease:[0.4, 0, 0.2, 1] }
```

5.2 Core Component Specifications

Button

```
// Button variants
// primary: brand-500 bg, white text, brand-600 hover
// secondary: white bg, brand-500 border+text, brand-50 hover
// ghost: transparent, brand-500 text, brand-50 hover
// danger: error bg, white text
// icon: square, icon only, ghost style

// Sizes: sm (32px h), md (40px h, default), lg (48px h)

// Motion: scale(0.97) on press, 100ms spring
// Loading state: spinner replaces text, button disabled
// All buttons: focus ring --shadow-focus, border-radius --radius-md

// Example React usage (custom component, no library):
<Button variant='primary' size='md' loading={isSubmitting}>
  Save Changes
</Button>
```

Data Table

```
// Features required in every data table:
// - Column sorting (click header)
// - Column visibility toggle (show/hide columns)
// - Row selection (checkbox, bulk actions)
// - Pagination (page size: 20/50/100)
```

```
// - Inline search/filter
// - Export to CSV/PDF
// - Sticky header on scroll
// - Empty state (illustration + CTA)
// - Loading skeleton (matches exact table structure)

// Dynamic columns: built from MetaSchema fields
// Only renders columns the current role has READ permission for
const columns = metaSchema.tables
  .find(t => t.tableName === 'students')
  .fields
  .filter(f => canRead(permissions, 'students', f.id))
  .map(f => ({ key: f.columnName, label: t(f.id) }));
```

Canvas Node Design

```
// Role Node visual spec
// Dimensions: 200px × 80px minimum, expands with content
// Border: 1.5px solid var(--color-brand-200)
// Border-radius: var(--radius-lg)
// Background: white, brand-50 on hover, brand-100 when selected
// Shadow: --shadow-md on hover, --shadow-lg when dragging
// Header: brand-500 bg strip 8px tall at top (color-coded by role type)
// Content: role name (semibold), permission count (muted)
// User count badge: top-right corner, accent-500 bg

// Connection line spec
// Type: smooth bezier (React Flow 'smoothstep' edge)
// Color: brand-200 default, brand-500 on hover/selected
// Width: 2px
// Arrow: filled chevron at target node
// Label: gray-600 text on white pill, --radius-full, shadow-sm

// Drag behavior (Motion)
// dragConstraints: none (free canvas)
// dragMomentum: false
// onDragEnd: snap to grid (20px grid)
// drag animation: shadow-lg + scale(1.02) while dragging
```

5.3 Page Layout System

```
// App shell layout
// Left sidebar: 240px fixed, collapsible to 64px (icons only)
// Top bar: 56px fixed, breadcrumb + global search + user menu + AI agent toggle
// Main content: fluid, max-width 1440px, padding var(--space-8)
// Right panel: 320px slide-in (AI agent chat, record detail)

// Sidebar structure:
// - School logo + name
// - Navigation items (icon + label)
// - Section groups (Academic, Finance, HR, Operations, Settings)
// - Active item: brand-50 bg, brand-500 left border 3px
```

```
// - Hover: gray-50 bg, 200ms smooth transition
// - Collapse animation: width 240-64px, 300ms smooth ease

// Top bar structure:
// - Breadcrumb: Home / Module / Page
// - Global search (Cmd+K): modal overlay, fuzzy search across all modules
// - Notification bell: badge count, dropdown list
// - Language switcher: flag + code
// - User avatar: initials circle, dropdown (profile, settings, logout)
// - AI agent button: animated pulse when agent has a proactive suggestion
```

6 AI Incident Intelligence System

The AI Incident Intelligence System (AIIS) is a platform-level subsystem that monitors all school deployments, detects issues across the entire stack, performs AI-powered root cause analysis, and delivers plain-English incident reports to the Company Owner Portal and mobile alerts. It operates continuously and silently — the owner is never expected to dig through logs.

Design Principle

The AIIS is not a log aggregator. It is an intelligent diagnosis system. Its job is to answer: 'Something broke — what exactly happened, why did it happen, which schools are affected, how severe is it, and what should we do right now?' The owner receives a clear answer, not a wall of stack traces.

6.1 Detection Layer — What It Monitors

Signal	Detection Method
API error rate spike	Fastify error rate >5% over 2-min window per school. Threshold per endpoint. Detected via Prometheus metrics.
Schema migration failure	Any schema change that fails or partially applies. Detected via migration log watcher.
Provisioning stuck	Any school in non-terminal provision state for >30 minutes. Detected via Control Plane cron.
Database connection exhaustion	Cloud SQL connection count >90% of pool. Detected via GCP monitoring metric.
Memory/CPU spike	Cloud Run instance CPU >85% sustained for >5 minutes. Detected via GCP Cloud Monitoring.
Canvas save failure	Canvas save returning errors for >3 consecutive attempts by any user. Detected via API logs.
AI agent tool error	Claude API returning errors or tool execution failures. Detected via agent error log.
Slow query detection	Any query taking >1 second logged via pg_stat_statements. Aggregated and alerted if affecting >10% of requests.
Billing/subscription anomaly	School subscription expires without renewal. Detected via Control Plane cron.
Mobile crash rate	React Native crash rate >0.1% of sessions. Detected via Sentry mobile SDK.
Security anomaly	Unusual access pattern: >50 failed auth attempts, cross-tenant probe detected, abnormal API volume from single IP.

6.2 Root Cause Analysis Algorithm

When a signal fires, the AIIS assembles a context package and sends it to the Claude API for root cause analysis. The agent has access to read (never write) from: API error logs, slow query logs, migration history, provisioning logs, Redis memory stats, and GCP monitoring metrics.

```
// services/aiis/analyze.ts

export async function analyzeIncident(signal: IncidentSignal):
Promise<IncidentReport> {
  // 1. Gather evidence from all available sources
  const evidence = await gatherEvidence(signal);
  // evidence = { errorLogs, slowQueries, recentMigrations,
  //             redisStats, gcpMetrics, recentDeployments }

  // 2. Build analysis prompt for Claude
  const prompt = buildAnalysisPrompt(signal, evidence);

  // 3. Call Claude API for root cause analysis
  const response = await anthropic.messages.create({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 1500,
    system: `You are an expert platform reliability engineer analyzing an
            incident on a multi-tenant school management SaaS platform.
            Analyze the provided evidence and return a JSON incident report.`,
    messages: [{ role: 'user', content: prompt }]
  });

  // 4. Parse structured report from Claude response
  const report = parseIncidentReport(response.content[0].text);

  // 5. Store report and trigger notifications
  await storeIncidentReport(report);
  await notifyOwner(report);

  return report;
}

function buildAnalysisPrompt(signal: IncidentSignal, ev: Evidence): string {
  return `
INCIDENT SIGNAL:
Type: ${signal.type}
Severity: ${signal.severity}
Affected school(s): ${signal.schoolIds.join(', ')}
Detected at: ${signal.detectedAt}

EVIDENCE:
Recent error logs (last 50): ${JSON.stringify(ev.errorLogs)}
Slow queries detected: ${JSON.stringify(ev.slowQueries)}
Recent schema migrations: ${JSON.stringify(ev.recentMigrations)}
Recent deployments: ${JSON.stringify(ev.recentDeployments)}
Resource metrics: ${JSON.stringify(ev.gcpMetrics)}`
}
```

```
Return a JSON object with these exact fields:
{ rootCause, severity, affectedSchools, affectedUsers,
  technicalDetail, ownerSummary, immediateAction, longTermFix,
  isAutoResolvable, estimatedImpact }
.trim();
}
```

6.3 Incident Report Structure

```
interface IncidentReport {
  id: string;
  detectedAt: string;
  resolvedAt: string | null;
  severity: 'P1_CRITICAL' | 'P2_HIGH' | 'P3_MEDIUM' | 'P4_LOW';
  type: string;
  affectedSchools: string[];
  affectedUsers: number; // estimated count

  // Plain English – what the owner reads
  ownerSummary: string; // e.g. 'Fee payment module is failing for
3 schools'
  rootCause: string; // e.g. 'Schema migration 047 left a NULL
constraint'
  immediateAction: string; // e.g. 'Roll back migration 047 on
affected schools'
  longTermFix: string; // e.g. 'Add NULL check to migration
validator'

  // Technical detail – for engineering team
  technicalDetail: string;
  stackTrace?: string;
  slowQueries?: string[];

  isAutoResolvable: boolean; // can the system fix this without human
action?
  autoResolved: boolean;
  estimatedImpact: string; // e.g. 'Blocking fee collection for ~450
students'
}
```

6.4 Notification Delivery

Severity	Delivery Method
P1 CRITICAL	Immediate: push notification to owner's mobile (high-priority APNs/FCM), SMS to registered mobile number, Slack/WhatsApp message to ops channel. Portal shows full-screen alert banner.
P2 HIGH	Immediate: push notification, Slack alert. Portal shows red banner.
P3 MEDIUM	Within 5 minutes: push notification, portal notification badge. No SMS.
P4 LOW	Portal notification only. Batched into hourly digest if owner has that preference.

Auto-resolved	If AIIS auto-resolves: owner notified with 'This was automatically fixed' message + summary of what happened and what was done.
---------------	---

6.5 Auto-Resolution Capabilities

The AIIS can automatically resolve a subset of issues without human intervention. All auto-resolutions are logged and reported to the owner immediately after.

- — if single instance has memory leak (detected via metrics), AIIS triggers Cloud Run instance replacement. New instance starts, traffic shifts, old instance terminated. API Cloud Run instance restart
- — if Redis memory >90% causing evictions, AIIS flushes non-critical cache keys (permission cache, computed summaries). Core session data is never flushed. Redis cache flush
- — if DB connection pool exhausted due to leaked connections, AIIS restarts the API deployment (zero-downtime rolling restart via Cloud Run). Connection pool reset
- — if a specific query is detected as a repeat offender (>10 slow executions in 1 hour), AIIS creates a Redis cache layer for that query result with a 5-minute TTL and alerts engineering to add a proper index. Slow query cache

7 Company Owner Portal

The Company Owner Portal is a separate, internally-facing web application built on Next.js 16. It is deployed on a separate Cloud Run service with its own authentication — completely isolated from the school-facing product. Every member of the company team gets access based on their role.

7.1 Portal Roles & Access

Role	Access Level
OWNER	Full access to everything. Can see all schools, all financials, all incidents, execute any action including database access, schema rollback, and refunds. Can create/manage other portal users.
OPS_ENGINEER	Full technical access: provisioning, incident management, database queries, deployment triggers, school suspension/unsuspension. Cannot see billing revenue or issue refunds.
SUPPORT	Read access to school data to help with support tickets. Can view (not modify) any school's configuration, user list, and error logs. Cannot access financials or execute deployments.
FINANCE	Billing dashboard, subscription management, refund issuance, revenue analytics. Cannot access school data or technical systems.
SALES	School pipeline, trial management, subscription tier view. Read-only on school info. No technical or financial access.

7.2 Portal Dashboard — Main View

The portal home screen is a real-time operational dashboard showing the health of the entire platform at a glance.

Metric	What Is Shown
Total active schools	Live count. Breakdown by tier (Starter/Growth/Enterprise). Month-over-month growth rate.
Platform health score	Aggregate of API uptime, DB health, error rates across all schools. Green/amber/red RAG status.
Active incidents	Count of open P1/P2 incidents. Clicking opens the incident feed.
Revenue MTD	Current month's collected subscription revenue in INR. vs last month.
Schools in provisioning	Any schools currently being provisioned. Expected completion time.
AI agent activity	Total AI agent queries today across all schools. Cost estimate.
Recent deploys	Last 5 deployments with status (success/failed). Rollback button on each.
Resource cost today	GCP spend today vs budget. Alert if >120% of daily target.

7.3 School Detail View

Clicking any school opens a full detail view with complete information and all available actions.

Panel	Contents
School profile	Name, tier, region, branch count, student count, staff count, subscription status, provisioned date.
Resource health	Cloud SQL CPU/memory/disk, Redis memory, Cloud Run instance count. Real-time metrics pulled from GCP.
Recent incidents	Last 10 incidents for this school with severity, timestamp, root cause summary, resolution status.
Schema history	Last 20 canvas changes made by the school. Each entry shows: who made the change, what changed, timestamp. Rollback button on each entry.
Migration status	Which core schema version this school is on. Whether any pending migrations need to be applied.
User list	All users in this school with their roles. Support can reset passwords, ops can force-logout all sessions.
Database query console	OWNER and OPS_ENGINEER can execute read-only SQL queries against this school's database. All queries logged. Write queries require two-person approval.
Action panel	Suspend school, unsuspend, trigger manual schema migration, restart school's API context, clear Redis cache, issue refund (OWNER/FINANCE only).

7.4 Incident Feed

The incident feed is the AIIS output — a real-time stream of all detected issues, AI-generated root cause reports, and resolution status.

```
// Incident feed entry display

// Each incident card shows:
// ┌ [P1] Fee payment module failure           12 min ago
// │ 3 schools affected · ~1,200 students impacted
// └
// ┌ ROOT CAUSE
// │ Schema migration 047 added a NOT NULL constraint on
// │ fee_collections.gateway_ref but existing rows have NULL
// │ values, causing INSERT failures on new fee payments.
// └
// ┌ IMMEDIATE ACTION REQUIRED
// │ Roll back migration 047 on affected schools
// │ [ROLLBACK NOW]  [VIEW AFFECTED SCHOOLS]  [DISMISS]
// └

// One-click rollback: opens confirmation modal showing:
```

```
// - Exact SQL to be executed
// - Number of affected schools
// - Estimated execution time
// - 'Type ROLLBACK to confirm' safeguard
```

7.5 Portal Tech Stack

Layer	Implementation
Framework	Next.js 16.1 — separate app in Turborepo monorepo (apps/owner-portal/).
Auth	Separate auth system from school product. Internal email + password + mandatory 2FA for all portal users. No SSO with school accounts.
Real-time	WebSocket connection to a dedicated portal namespace on the API server. Incident alerts and school health metrics push in real-time.
GCP API calls	Portal backend calls GCP Cloud Monitoring API, Cloud SQL Admin API, and Cloud Run Admin API directly for real-time resource metrics.
Database query tool	Portal backend proxies read-only SQL queries to school databases through a dedicated read-only service account. Query timeout: 10 seconds. Result size limit: 1000 rows.
Audit log	Every portal action (who did what to which school) logged to a tamper-proof audit table in the Control Plane DB. Retention: 2 years.
Deployment	Cloud Run, separate service from school-facing web app. Minimum instances: 1 (always warm). VPC-only ingress — accessible only from within company VPN or via Cloud IAP.

8 Cloud Strategy & Developer Workflow

This section documents the cloud platform decision, the local development setup, the one-command deploy workflow, and the AI-assisted development approach using Claude Code CLI alongside this planning chat.

8.1 Cloud Platform Decision — GCP

After evaluating all three major cloud providers against the specific requirements of this platform, GCP is the chosen cloud for the following reasons:

Factor	Reasoning	Impact
Automatic sustained-use discounts	GCP automatically applies 30% discounts on always-on workloads with no reservation required. AWS requires manual reserved instance purchases for equivalent savings.	Cost — HIGH IMPACT
AI workload pricing	GCP is 5–10% cheaper than AWS and Azure for AI workloads (2026 data). Since this platform runs Claude API + Sarvam AI + AIIS continuously, this directly reduces operating costs.	Cost — HIGH IMPACT
Vertex AI native integration	The AIIS and AI agent can use Vertex AI for model serving and fine-tuning without leaving GCP's network. No cross-cloud data transfer fees.	Technical — HIGH
Cloud Run serverless containers	Cloud Run scales to zero when idle (cost saving in dev/staging), scales to 1000+ instances under load automatically. Zero Kubernetes management overhead.	Technical — HIGH
Mumbai region (asia-south1)	Primary region for all school data. Lowest latency for Indian schools. Data residency in India by default — important for future DPDPA compliance.	Compliance — HIGH
GCP free credits — \$300	New GCP account gets \$300 free credits. Enough to build and test the entire provisioning pipeline and run 5–10 schools in dev/staging before any billing.	Cost — IMMEDIATE
Committed use discounts later	Once stable: 1-year CUD gives 28% savings, 3-year gives 46% savings in Mumbai region. Lock in after Year 1 when school count and resource usage is predictable.	Cost — FUTURE
Terraform provider maturity	GCP's Terraform provider is comprehensive and well-	Technical — HIGH

	documented. The entire provisioning pipeline is written as Terraform modules — no manual GCP console clicks required.	
--	---	--

AWS & Azure — Why Not

AWS: Larger ecosystem but no automatic sustained-use discounts, slightly higher compute costs for always-on workloads, and no Vertex AI equivalent for tight Claude API integration. Azure: Best for enterprises with existing Microsoft licenses (Azure Hybrid Benefit). We have no Microsoft licenses — Azure's main cost advantage doesn't apply. Both are valid long-term options if the business strategy shifts (e.g., government contracts requiring Azure). Architecture is cloud-agnostic — switching later requires only config changes.

8.2 Local Development Setup — Mac

Everything runs locally on your Mac using Docker Desktop. The local stack is an exact mirror of GCP production — same PostgreSQL version, same Redis version, same Node.js version. No surprises when you deploy.

Prerequisites

```
# Install these on your Mac – one-time setup

# 1. Homebrew (Mac package manager)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2. Node.js 22 (LTS)
brew install node@22

# 3. Docker Desktop – download from docker.com/products/docker-desktop
# Gives you: PostgreSQL 16, Redis 7, all running locally as containers

# 4. Claude Code CLI
npm install -g @anthropic-ai/claude-code

# 5. Google Cloud CLI (for deploying to GCP)
brew install google-cloud-sdk

# 6. Terraform (for infrastructure provisioning)
brew install terraform

# Verify everything
node --version      # v22.x.x
docker --version    # Docker version 27.x
claude --version    # Claude Code version
gcloud --version    # Google Cloud SDK
terraform --version # Terraform v1.x
```

Docker Compose — Local Stack

```
# docker-compose.yml (root of monorepo)
version: '3.9'
services:

  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: schoolplatform_dev
      POSTGRES_USER: dev_user
      POSTGRES_PASSWORD: dev_password
    ports: ['5432:5432']
    volumes: ['postgres_data:/var/lib/postgresql/data']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U dev_user']
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 5s

  # Optional: local email testing (catches all outgoing emails)
  mailpit:
    image: axllent/mailpit
    ports: ['1025:1025', '8025:8025'] # SMTP + web UI

volumes:
  postgres_data:
```

Environment Variables — Local

```
# apps/api/.env.local
NODE_ENV=development
DATABASE_URL=postgresql://dev_user:dev_password@localhost:5432/schoolplatform_dev
REDIS_URL=redis://localhost:6379
JWT_SECRET=dev-jwt-secret-change-in-production
ANTHROPIC_API_KEY=your-claude-api-key
SARVAM_API_KEY=your-sarvam-api-key
GCP_PROJECT_ID=your-gcp-project-id

# GCP credentials for local Terraform/provisioning testing
GOOGLE_APPLICATION_CREDENTIALS=~/.config/gcloud/application_default_credentials.json

# apps/web/.env.local
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_WS_URL=ws://localhost:4000
```

8.3 One-Command Local Start

```
# Start everything locally – runs the entire platform on your Mac
npm run dev

# This single command (defined in root package.json) runs:
# 1. docker compose up -d           → starts PostgreSQL + Redis
# 2. npx turbo dev                  → starts all apps in parallel:
#    - apps/api on localhost:4000   (Fastify API)
#    - apps/web on localhost:3000   (Next.js web app)
#    - apps/owner-portal on :3001   (Owner portal)
# 3. Watches all packages for changes – hot reload on every save

# Stop everything
npm run dev:stop # docker compose down + kills all processes

# Reset local database (wipe and re-seed)
npm run db:reset # drops DB, recreates, runs all migrations, seeds test data
```

8.4 One-Command Deploy to GCP

```
# Deploy to production – triggered by git push, or manually:
git add .
git commit -m 'feat: add fee payment module'
git push origin main
# ↑ This single push triggers GitHub Actions which:
# 1. Runs all tests (PR checks)
# 2. Builds Docker images
# 3. Runs DB migrations on all active schools
# 4. Deploys API to GCP Cloud Run
# 5. Deploys Web to GCP Cloud Run
# 6. Deploys Owner Portal to GCP Cloud Run
# 7. Runs smoke tests
# 8. Notifies Slack on success / auto-rolls back on failure

# Manual deploy (emergency / bypass CI)
npm run deploy:production

# Deploy only the API (faster for backend-only changes)
npm run deploy:api

# Total time from git push to live: ~3-5 minutes
```

8.5 Claude Code CLI — How to Use It

Claude Code is the primary development tool for building this project. It runs in your terminal, reads the entire codebase, writes files, executes commands, and manages git — all through natural language. You describe what you want, it does it.

Installation & Setup

```
# Install Claude Code
npm install -g @anthropic-ai/claude-code

# Start Claude Code in your project directory
cd school-platform # your monorepo root
claude

# Claude Code reads your entire codebase automatically
# It understands your file structure, package.json, TypeScript types,
# existing code patterns – everything
```

CLAUDE.md — Project Context File

Create a CLAUDE.md file in the root of your monorepo. Claude Code reads this on every session. It tells Claude about your project so you never have to re-explain context.

```
# CLAUDE.md (root of monorepo – Claude Code reads this automatically)

## Project: School Management Platform
A multi-tenant SaaS school management platform for India.
India-first, AI-native, 100% customizable via drag-and-drop canvas.

## Stack
- Frontend: Next.js 16.1 + React 19.2 + TypeScript + Tailwind (tokens only)
- Backend: Fastify v5 + TypeScript + Node.js 22
- Mobile: React Native 0.83
- Database: PostgreSQL 16 (per-school isolated) + Redis 7
- Cloud: GCP (Cloud Run + Cloud SQL + Cloud Storage + Secret Manager)
- AI: Claude API (claude-sonnet-4-6) + Sarvam AI for voice
- Monorepo: Turborepo

## Architecture
Two-plane model: Control Plane (our ops) + Data Plane (per-school isolated DB).
Schema Migration Engine: real-time DB schema changes triggered by canvas UI.
Canvas Builder: React Flow based role + workflow builder.

## Key Rules
- NEVER hardcode secrets – always use GCP Secret Manager or .env.local
- ALL DB queries must be scoped by school_id (tenant isolation)
- ALL column/table names from user input must pass [a-z0-9_] validation
- Use parameterized queries – never string interpolate user input into SQL
- Every new API route needs: auth middleware, permission check, rate limit
- TypeScript strict mode – no 'any' types

## Current Build Phase
Phase 1: Monorepo scaffold + Docker Compose + GCP project init + Control Plane DB

## Docs
Full architecture in /docs/TechnicalArchitecture_v1.docx
Operations reference in /docs/OperationsDesignReference_v1.docx
```


Example Claude Code Workflows

Task	What to Say to Claude Code
Scaffold a new API route	In terminal: 'Add a GET /api/v1/school/classes route that returns all classes for the current tenant, with pagination, sorted by name. Use the existing students route as a pattern.'
Fix a failing test	'Run the test suite and fix any failing tests in the schema engine package.'
Full feature build	'Build the attendance marking feature: API endpoint to mark daily attendance for a class, store in attendance table, publish a WebSocket event, and update the Redis cache key attendance:{classId}:{date}.'
Database migration	'Create a migration file 006_add_exam_schedules.sql that adds an exam_schedules table with the fields shown in the MetaSchema for the exams module.'
Debug an issue	'The school provisioning is failing at the SCHEMA_SEEDING step. Look at the provision_log table structure and the seedCoreSchema function and tell me what's going wrong.'
Deploy	'Run the full test suite, then if all tests pass, commit everything with message feat: add attendance module and push to main.'

8.6 Integrating This Chat with Claude Code

You asked if there is a way to connect this claude.ai planning chat with Claude Code in your terminal. Here is the honest current answer and the best workflow:

Current State (May 2026)

Claude Code (terminal) and claude.ai (this chat) are separate products running the same underlying models but with separate contexts. There is no direct live sync between a claude.ai conversation and a Claude Code terminal session. However, there is a highly effective bridge workflow that gives you 95% of what a native integration would provide.

The Bridge Workflow — How to Connect Them

```
# Method 1: CLAUDE.md as the bridge (recommended)
# Keep your CLAUDE.md updated with decisions from this chat.
# When we make architectural decisions here, paste the key points into CLAUDE.md.
# Claude Code reads CLAUDE.md on every session – it inherits all context.

# Method 2: Export conversation summaries as slash commands
# In Claude Code terminal, create custom slash commands:
mkdir -p ~/.claude/commands

# Create a slash command with the full tech spec:
cat > ~/.claude/commands/platform-context.md << 'EOF'
Read the full platform context from /path/to/project/docs/
and from CLAUDE.md before answering any questions.
```

```
This is a school management SaaS platform – full specs in those docs.
EOF

# Now in Claude Code, type:
/platform-context
# Claude Code loads all docs and has full context instantly

# Method 3: MCP (Model Context Protocol) – future integration
# Claude Code supports MCP servers. In theory, a custom MCP server
# could expose claude.ai conversation history to Claude Code.
# This is not a built-in feature today but is architecturally possible
# to build as a custom MCP server.
```

Recommended Daily Workflow

Tool	Use For
Planning & Architecture (claude.ai — this chat)	Use this chat for: system design decisions, brainstorming new modules, debugging complex architectural questions, reviewing what we've built, updating the docs. This chat has memory of everything we've discussed across sessions.
Building & Coding (Claude Code in terminal)	Use Claude Code for: writing actual code, running tests, fixing bugs, creating files, running migrations, committing and deploying. Point it at the docs we've created and CLAUDE.md for context.
Keeping them in sync	After key decisions in this chat → update CLAUDE.md. After building something in Claude Code → come back here to document it and plan the next piece. The two tools complement each other perfectly.
Claude Code with docs	Place all three docs (Brainstorm, Technical Architecture, Operations Reference) in a /docs folder in the monorepo. Claude Code can read them with: 'Read the technical architecture doc and implement the provisioning pipeline.'

Best Setup for Solo Building

Open two windows side by side: this claude.ai chat on the left for planning and architecture, Claude Code terminal on the right for building. Plan here, build there. When something in the code raises a question, ask it here. When we make a decision here, Claude Code implements it. This is the most powerful solo development workflow available in 2026.

8.7 GCP Project Initial Setup

Run these commands once to initialize your GCP project before the first deploy. Claude Code can execute all of these for you — just paste this section and say 'run the GCP setup'.

```
# 1. Create GCP project
gcloud projects create school-platform-prod --name='School Platform'
gcloud config set project school-platform-prod

# 2. Enable required APIs
gcloud services enable \
```

```
sqladmin.googleapis.com \  
redis.googleapis.com \  
run.googleapis.com \  
secretmanager.googleapis.com \  
storage.googleapis.com \  
cloudmonitoring.googleapis.com \  
cloudbilling.googleapis.com \  
artifactregistry.googleapis.com \  
compute.googleapis.com  
  
# 3. Create Artifact Registry for Docker images  
gcloud artifacts repositories create school-platform \  
  --repository-format=docker \  
  --location=asia-south1  
  
# 4. Create service account for GitHub Actions deploy  
gcloud iam service-accounts create github-deploy \  
  --display-name='GitHub Actions Deploy'  
  
gcloud projects add-iam-policy-binding school-platform-prod \  
  --member='serviceAccount:github-deploy@school-platform-  
prod.iam.gserviceaccount.com' \  
  --role='roles/run.admin'  
  
# 5. Export service account key for GitHub Actions secret  
gcloud iam service-accounts keys create github-sa-key.json \  
  --iam-account=github-deploy@school-platform-prod.iam.gserviceaccount.com  
# → Upload contents of github-sa-key.json as GCP_SA_KEY in GitHub repo secrets  
  
# 6. Authenticate local development  
gcloud auth application-default login  
# → Opens browser, sign in, credentials saved to ~/.config/gcloud/  
  
# That's it. GCP is ready. Now run: npm run dev
```

Document Complete — 8 Sections

This document now covers 8 complete operational and design specifications: Database Migration Runbook, CI/CD Pipeline, Security Audit Checklist, OpenAPI Contract, UI/UX Design Specification, AI Incident Intelligence System, Company Owner Portal, and Cloud Strategy & Developer Workflow (GCP decision rationale with 2026 pricing data, local Docker setup, one-command deploy, Claude Code CLI setup and CLAUDE.md template, chat-to-terminal bridge workflow, and GCP project initialization commands).

Combined with the Brainstorm Document and Technical Architecture Document, this is the complete pre-build documentation suite. You now have everything needed to open Claude Code in your terminal, point it at these docs, and start building the platform from day one.